REST APIs (ASP.NET CORE WEB API-Senior .NET Interview)

Interviewer Perspective (8+ Years):

For a senior .NET developer, I don't expect just definitions like "REST stands for Representational State Transfer." I expect you to explain **REST principles, API design, security, performance, scalability, versioning, and production best practices**.

Question 1: What is REST API?

Interviewer

What is a REST API? Explain it with a real-world example.

Answer

REST (Representational State Transfer) is an **architectural style** for designing web services that communicate over HTTP.

A REST API exposes **resources** using **URIs** and manipulates them using standard HTTP methods.

Example:

Employee Resource

/api/employees

Instead of calling

GetEmployee()

SaveEmployee()

DeleteEmployee()

REST uses HTTP methods.

HTTP Method	URI	Action
GET	/api/employees	Get all employees
GET	/api/employees/10	Get employee by ID
POST	/api/employees	Create employee
PUT	/api/employees/10	Update employee

HTTP Method	URI	Action
DELETE	/api/employees/10	Delete employee

Real-Time Example

Insurance Policy API

GET

/api/policies/100

Returns

```
{
  "policyId":100,
  "customer":"Satheesh",
  "premium":15000
}
```

REST Principles

REST has six constraints.

1. Client-Server

Client and Server are independent.

Angular

↓

Web API

↓

SQL Server

---

2. Stateless

Every request contains all required information.

Server does not remember previous requests.

Example

Every request sends

Authorization

Bearer Token

---

3. Cacheable

Responses may be cached.

GET /countries

Can be cached.

---

4. Uniform Interface

Resources should have consistent URLs.

Good

/api/employees

Bad

/GetEmployeeData

/DeleteEmployeeData

---

5. Layered System

Client

↓

API Gateway

↓

Web API

↓

Database

---

6. Code on Demand (Optional)

Server can send executable code (rarely used in modern APIs).

Interview Question

Is REST a protocol?

Answer

No.

REST is an architectural style.

HTTP is the protocol.

Question 2

Explain RESTful API

Very commonly asked.

Answer

A RESTful API follows REST principles.

Example

Good

GET

/api/orders/100

Bad

GET

/GetOrderDetails?id=100

Characteristics

✓ Resource Based

✓ Stateless

✓ Cacheable

✓ HTTP Methods

✓ Proper Status Codes

✓ Uniform URI

Interview Tip

Don't say

"REST means using GET and POST."

That is incorrect.

REST is about architectural principles.

Question 3

Explain HTTP Methods

One of the most frequently asked questions.

GET

Retrieve data.

Example

GET /api/employees

Should not modify data.

Safe

Yes

Idempotent

Yes

POST

Creates a resource.

POST /api/employees

Request

{

"name":"Satheesh"

}

Response

201 Created

PUT

Replaces the entire resource.

PUT

/api/employees/10

PATCH

Updates part of the resource.

PATCH

/api/employees/10

Example

Only Salary

DELETE

Deletes resource.

DELETE

/api/employees/10

Interview Question

Difference

PUT

vs

PATCH

PUT	PATCH
Full Update	Partial Update
Replaces Resource	Updates Selected Fields
Usually sends entire object	Sends only changed fields

Question 4

Explain HTTP Status Codes

Extremely important.

2xx Success

Code	Meaning
200	OK
201	Created
202	Accepted
204	No Content

4xx Client Error

Code	Meaning
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
409	Conflict
422	Unprocessable Content (formerly Entity)

5xx Server Error

Code	Meaning
500	Internal Server Error
502	Bad Gateway
503	Service Unavailable
504	Gateway Timeout

Senior Interview Question

Difference

401

and

403

Answer

401

↓

User not authenticated.

403

↓

User authenticated

↓

No permission.

Question 5

Explain Idempotent Methods

Most asked.

Answer

An operation is **idempotent** if performing it multiple times produces the same end state as performing it once.

Example

DELETE

/api/employees/10

Call

Once

↓

Employee Deleted

Call Again

↓

Still Deleted (or 404 if already gone, depending on API design)

Final state remains the same.

Idempotent Methods

Method	Idempotent
GET	Yes
PUT	Yes
DELETE	Yes (state remains unchanged after the first successful delete)
HEAD	Yes
OPTIONS	Yes
POST	No

Interview Question

Why is POST not idempotent?

Answer

Each request can create a new resource.

Example

POST

/api/orders

Calling

5 Times

↓

Creates

5 Orders

Question 6

Explain Safe Methods

Safe methods

Do not modify server data.

Method	Safe
GET	Yes
HEAD	Yes
OPTIONS	Yes
POST	No
PUT	No
DELETE	No

Interview Question

Is GET always safe?

Answer

It **should be**.

A properly designed REST API should not modify server state in response to a GET request.

Question 7

Explain URI Design Best Practices

Interview Favorite.

Good

/api/customers

/api/customers/100

/api/customers/100/orders

Bad

/GetCustomer

/DeleteCustomer

Best Practices

✓ Use nouns

✓ Use plural names

✓ Lowercase URLs

✓ Avoid verbs

✓ Keep URLs meaningful

Question 8

Explain Stateless APIs

Every request contains

Everything Needed

Example

Authorization

Bearer Token

Every request includes the token because the server does not rely on session state.

Advantages

✓ Scalability

✓ Load Balancing

✓ Easy Deployment

Question 9

Explain Resource Naming

Good

/api/products

Good

/api/products/10

Good

/api/products/10/reviews

Bad

/GetProducts

Question 10

Explain CRUD Mapping

CRUD	HTTP
Create	POST
Read	GET
Update	PUT/PATCH
Delete	DELETE

Interview Scenario ⭐

Interviewer

Design APIs for an Employee System.

Expected Answer

GET

/api/employees

GET

/api/employees/{id}

POST

/api/employees

PUT

/api/employees/{id}

PATCH

/api/employees/{id}

DELETE

/api/employees/{id}

Module 4 Progress

We've covered:

- ✅ REST Principles
- ✅ RESTful APIs
- ✅ HTTP Methods
- ✅ HTTP Status Codes
- ✅ Idempotency
- ✅ Safe Methods
- ✅ URI Design
- ✅ Stateless APIs
- ✅ Resource Naming
- ✅ CRUD Mapping

Next Part (Module 4 – Part 2)

We'll cover advanced REST topics that are frequently asked in senior interviews:

1. API Versioning
2. Pagination
3. Filtering

4. Sorting
5. Searching
6. HATEOAS
7. Content Negotiation
8. Swagger/OpenAPI
9. API Documentation
10. Error Handling
11. ETags & Optimistic Concurrency
12. Caching in REST APIs
13. API Security Best Practices
14. OAuth 2.0 vs JWT
15. Real-world API design scenarios

Recommendation for interview success: After we complete the REST API module, we'll move to **Azure PaaS & Azure Cloud**, which is one of the highest-weighted areas for senior .NET developer interviews, especially given your experience with Azure, Managed Identity, Azure DevOps, Cosmos DB, and Bicep.

Excellent. Now we're entering the **Advanced REST API** topics. These questions are commonly asked in interviews at **Microsoft, Deloitte, EY, Accenture, Cognizant, Capgemini, IBM, TCS Digital**, and other companies for **8–15 years** experience.

Senior Interviewer's Expectation: You should be able to design APIs that are **secure, scalable, maintainable, versioned, and performant**, not just explain HTTP methods.

Module 4 – REST APIs

Part 2 (Advanced REST APIs)

Question 11: How do you implement API Versioning?

Interviewer

Your API is already used by thousands of clients. A new requirement changes the response structure. How do you handle it without breaking existing clients?

Answer

API Versioning allows us to introduce changes while maintaining backward compatibility.

Instead of modifying the existing API:

GET /api/employees

Create a new version.

Versioning Approaches

1. URL Versioning (Most Common)

GET /api/v1/employees

GET /api/v2/employees

Advantages

- Easy to understand
- Easy to test
- Easy to document

---

2. Query String Versioning

GET /api/employees?version=1

---

3. Header Versioning

GET /api/employees

Header

api-version: 2

---

4. Media Type Versioning

Accept: application/json;version=2

Interview Question

Which versioning strategy do you recommend?

Senior Answer

For enterprise APIs, **URL versioning** is commonly preferred because it is explicit, easy to document, and works well with API gateways and client tools. Some organizations prefer header or media-type versioning depending on their API standards.

Question 12

Explain Pagination

One of the most frequently asked REST API interview questions.

Problem

Imagine the Employee table contains:

50,00,000 Records

Returning all records:

GET /api/employees

Problems

- Slow response
- High memory usage
- Network overhead
- Poor user experience

Solution

Pagination

GET /api/employees?page=2&pageSize=20

Response

```
{
  "page":2,
  "pageSize":20,
  "totalRecords":5000000,
  "data":[]
}
```

SQL Example

```
SELECT *
FROM Employees
ORDER BY EmployeeId
OFFSET 20 ROWS
FETCH NEXT 20 ROWS ONLY;
```

Best Practices

✓ Maximum Page Size

✓ Include Total Count when appropriate

✓ Return paging metadata

Interview Question

Should page size be unlimited?

Answer

No.

Example

Maximum

100

Question 13

Explain Filtering

Suppose

Employee Table

contains

100000 Employees.

Need

Only HR Department.

Instead of

GET /api/employees

Use

GET /api/employees?department=HR

Multiple Filters

GET /api/employees?department=HR&city=Hyderabad

Advantages

- Faster
- Less network traffic
- Better user experience

Question 14

Explain Sorting

Without Sorting

Random Order.

Sort by Salary

GET /api/employees?sort=salary

Descending

GET /api/employees?sort=-salary

or

GET /api/employees?sort=salary_desc

(depending on your API design)

Multiple Sort

sort=department,name

Question 15

Explain Searching

Example

Search Employee

GET /api/employees?search=satheesh

Advanced Search

GET /api/employees

?department=IT

&city=Hyderabad

&experience=8

Interview Tip

Searching

↓

Filtering

↓

Sorting

↓

Pagination

is a common combination in enterprise APIs.

Question 16

Explain HATEOAS

Senior-level interview question.

HATEOAS

means

Hypermedia As The Engine Of Application State

Response contains

Data

-

Links

Example

```
{
 "employeeId":10,
 "name":"Satheesh",
 "links":
 [
  {
    "rel":"update",
    "href":"/employees/10"
  },
  {
    "rel":"delete",
    "href":"/employees/10"
  }
 ]
}
```

Advantages

- Discoverability
- Loose coupling
- Easier API evolution

Interview Question

Is HATEOAS mandatory?

Answer

No.

Many public APIs do not implement it because it increases complexity.

Question 17

Explain Content Negotiation

Very common.

Client sends

Accept: application/json

Server returns

JSON

Client sends

Accept: application/xml

Server

↓

XML

(if configured)

ASP.NET Core

supports

JSON

by default.

Interview Question

Why is JSON preferred?

Answer

- Smaller payload
- Easy parsing
- Excellent browser and mobile support
- Widely adopted

Question 18

Explain Swagger / OpenAPI

Every interview asks this.

Swagger

provides

Interactive API Documentation.

Registration

builder.Services.AddEndpointsApiExplorer();

builder.Services.AddSwaggerGen();

Pipeline

app.UseSwagger();

app.UseSwaggerUI();

Advantages

✓ Documentation

✓ API Testing

✓ Client SDK Generation

✓ Easy Integration

Interview Question

Should Swagger be enabled in Production?

Senior Answer

Generally, **Swagger UI is disabled or restricted** in production unless there is a business need. Public exposure should be controlled using authentication, IP restrictions, or environment-based configuration.

Question 19

Explain API Error Handling

Good API

Returns

Consistent Errors.

Example

{

"status":404,

"title":"Employee Not Found"

}

Instead of

Object reference not set...

Best Practice

Use

Global Exception Middleware

-

ProblemDetails

Question 20

Explain ETag (Optimistic Concurrency)

Very popular in product companies.

Problem

Two users edit

Same Employee.

User A

↓

Updates Salary

User B

↓

Updates Address

Without concurrency

↓

One update may overwrite the other.

Solution

ETag

Server returns

ETag: "v1"

Client sends

If-Match: "v1"

If version changed

↓

Return

412 Precondition Failed

Advantages

✓ Prevents Lost Updates

✓ Better Concurrency

Question 21

REST API Caching

Example

Countries

States

Cities

Rarely Change

Use

Cache-Control

max-age=3600

Benefits

✓ Faster

✓ Less Database Load

✓ Better User Experience

Question 22

OAuth 2.0 vs JWT

Very frequently asked.

OAuth	**JWT**
Authorization Framework	Token Format
Grants delegated access	Carries claims
Works with Identity Providers	Used after authentication
Defines authorization flows	Often used as the access token format

---

Interview Question

Can OAuth use JWT?

Answer

Yes.

Many OAuth implementations issue **JWT access tokens**, although opaque tokens are also possible.

Question 23

REST API Security Best Practices

Always use

✓ HTTPS

✓ JWT/OAuth

✓ Input Validation

✓ Rate Limiting

✓ CORS

✓ Parameterized SQL

✓ Secure Headers

✓ Logging & Monitoring

✓ Avoid Sensitive Data in Responses

✓ Principle of Least Privilege

Question 24

Scenario-Based Question ⭐

Interviewer

Your GET API takes **12 seconds** to return data.

How would you optimize it?

Strong Senior Answer

I would investigate in this order:

1. Database

- Check execution plan
- Verify indexes
- Identify blocking or deadlocks
- Optimize queries

---

2. API

- Use asynchronous calls
- Reduce unnecessary object mapping
- Avoid N+1 query problems
- Return only required fields (DTOs)

---

3. Network

- Enable compression
- Use pagination
- Use caching where appropriate

---

4. Infrastructure

- Review CPU and memory usage
- Check connection pool utilization

- Analyze Application Insights or other telemetry

---

5. External Dependencies

- Measure latency of downstream REST APIs
- Add retries and timeouts where appropriate
- Consider circuit breakers for resiliency

Question 25

Design a REST API for an Insurance System

Interviewer

Design REST APIs for managing insurance policies.

Strong Answer

GET /api/policies

Get all policies.

GET /api/policies/{id}

Get a policy by ID.

POST /api/policies

Create a policy.

PUT /api/policies/{id}

Replace a policy.

PATCH /api/policies/{id}

Update part of a policy.

DELETE /api/policies/{id}

Delete a policy.

GET /api/policies?page=1&pageSize=20&status=Active&sort=policyNumber

Supports filtering, sorting, and pagination.

Top 15 Rapid-Fire REST API Questions

Q1. Is REST a protocol?

Answer: No, it is an architectural style.

Q2. What makes an API RESTful?

Answer: It follows REST constraints such as statelessness, resource-based URIs, and proper use of HTTP methods.

Q3. Difference between PUT and PATCH?

Answer: PUT replaces the entire resource; PATCH updates part of it.

Q4. Why is GET idempotent?

Answer: Repeating a GET request should not change the server's state.

Q5. Why use DTOs?

Answer: To avoid exposing domain entities directly, reduce payload size, and shape responses for clients.

Q6. What is HATEOAS?

Answer: A REST constraint where responses include hyperlinks to related actions.

Q7. What is Content Negotiation?

Answer: Selecting the response format based on the client's Accept header.

Q8. Why use Swagger?

Answer: Interactive API documentation and testing.

Q9. What is ETag?

Answer: A mechanism for optimistic concurrency and cache validation.

Q10. Difference between 401 and 403?

Answer: 401 means authentication is required or failed; 403 means the user is authenticated but not authorized.

Q11. What is API Versioning?

Answer: Supporting multiple API versions without breaking existing clients.

Q12. Why use Pagination?

Answer: To improve performance and reduce payload size.

Q13. Why use Rate Limiting?

Answer: To protect APIs from abuse and improve availability.

Q14. What is Statelessness?

Answer: Each request contains all the information required to process it.

Q15. Why prefer HTTPS?

Answer: It encrypts communication, protecting data in transit and ensuring integrity.

Module 4 Completed

You now have senior-level coverage of:

- REST Architecture
- REST Constraints
- HTTP Methods
- HTTP Status Codes
- Idempotency
- URI Design
- Versioning
- Pagination
- Filtering

- Sorting
- Searching
- HATEOAS
- Content Negotiation
- Swagger/OpenAPI
- Error Handling
- ProblemDetails
- ETags & Optimistic Concurrency
- REST Security
- Performance Optimization
- API Design Scenarios
- Rapid-Fire Questions

Next Module: Azure PaaS & Azure Cloud (Most Important)

Given your background and the demand in senior .NET roles, the next module will cover:

- Azure App Service
- Azure Functions
- Azure Storage (Blob, Queue, Table, File)
- Azure Service Bus
- Azure Event Grid
- Azure Event Hub
- Azure Key Vault
- Azure Managed Identity
- Azure Cosmos DB
- Azure SQL Database
- Azure API Management (APIM)
- Azure Monitor & Application Insights
- Azure Networking
- Private Endpoints
- Azure RBAC
- Real-world architecture and troubleshooting scenarios

This is one of the most valuable modules for senior .NET and Azure interviews.